

3.8: Performing Subqueries

Ben Diemler

Step 1:

1- Subquery 3.7:

```
SELECT A.customer_id,  
A.first_name,  
A.last_name,  
C.city,  
D.country,  
SUM(amount) AS total_amount_paid  
FROM payment E  
INNER JOIN customer A ON E.customer_id = A.customer_id  
INNER JOIN address B ON A.address_id = B.address_id  
INNER JOIN city C ON B.city_id = C.city_id  
INNER JOIN country D ON C.country_id = D.country_id  
WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule (Dhulia)',  
'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')  
GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country  
ORDER BY total_amount_paid DESC  
LIMIT 5;
```

2- Outer statement:

```
SELECT AVG (total_amount_paid) AS avg_payment_amount  
FROM subquery
```

3- Final query:

```
SELECT AVG(total_amount_paid) AS avg_payment_amount  
FROM  
(SELECT A.customer_id,  
A.first_name,  
A.last_name,  
C.city,  
D.country,  
SUM(amount) AS total_amount_paid  
FROM payment E  
INNER JOIN customer A ON E.customer_id = A.customer_id  
INNER JOIN address B ON A.address_id = B.address_id  
INNER JOIN city C ON B.city_id = C.city_id  
INNER JOIN country D ON C.country_id = D.country_id  
WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule (Dhulia)',  
'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')  
GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country  
ORDER BY total_amount_paid DESC  
LIMIT 5) AS average
```

4- Data output:

```
"avg_payment_amount"  
107.3540000000000000
```

Query Query History

```

1 SELECT AVG(total_amount_paid) AS avg_payment_amount
2 FROM
3 (SELECT A.customer_id,
4 A.first_name,
5 A.last_name,
6 C.city,
7 D.country,
8 SUM(amount) AS total_amount_paid
9 FROM payment E
10 INNER JOIN customer A ON E.customer_id = A.customer_id
11 INNER JOIN address B ON A.address_id = B.address_id
12 INNER JOIN city C ON B.city_id = C.city_id
13 INNER JOIN country D ON C.country_id = D.country_id
14 WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule (Dhulia)',
15 'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
16 GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country
17 ORDER BY total_amount_paid DESC
18 LIMIT 5) AS average
19
20

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1

avg_payment_amount
numeric
107.3540000000000000

Step 2:

1- Query step 3.7: Inner query

```

(SELECT A.customer_id,
A.first_name,
A.last_name,
C.city,
D.country,
SUM(amount) AS total_amount_paid
FROM payment E
INNER JOIN customer A ON E.customer_id = A.customer_id
INNER JOIN address B ON A.address_id = B.address_id
INNER JOIN city C ON B.city_id = C.city_id
INNER JOIN country D ON C.country_id = D.country_id
WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule (Dhulia)',
'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country
ORDER BY total_amount_paid DESC
LIMIT 5;)

```

2- Outer query:

```

SELECT country,
COUNT (customer_id) AS all_customer_count
FROM country
INNER JOIN city ON city.country_id = country.country_id
INNER JOIN address ON address.city_id = city.city_id
INNER JOIN customer ON customer.address_id = address.address_id
GROUP BY country
ORDER BY all_customer_count DESC

```

3- Final query:

```

SELECT ctr.country,
COUNT (DISTINCT cst.customer_id) AS all_customer_count,
COUNT(DISTINCT top_5_customers.customer_id) AS top_customer_count
FROM customer AS cst

```

```

INNER JOIN address AS adr ON cst.address_id = adr.address_id
INNER JOIN city AS cty ON adr.city_id = cty.city_id
INNER JOIN country AS ctr ON cty.country_id = ctr.country_id
LEFT JOIN
(SELECT A.customer_id,
A.first_name,
A.last_name,
C.city,
D.country,
SUM(amount) AS total_amount_paid
FROM payment E
INNER JOIN customer A ON E.customer_id = A.customer_id
INNER JOIN address B ON A.address_id = B.address_id
INNER JOIN city C ON B.city_id = C.city_id
INNER JOIN country D ON C.country_id = D.country_id
WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule (Dhulia)',
'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country
ORDER BY total_amount_paid DESC
LIMIT 5)
AS top_5_customers
ON top_5_customers.country = ctr.country
GROUP BY ctr.country
ORDER BY top_customer_count DESC, all_customer_count DESC

```

Query	Query History
1	SELECT ctr.country,
2	COUNT (DISTINCT cst.customer_id) AS all_customer_count,
3	COUNT (DISTINCT top_5_customers.customer_id) AS top_customer_count
4	FROM customer AS cst
5	INNER JOIN address AS adr ON cst.address_id = adr.address_id
6	INNER JOIN city AS cty ON adr.city_id = cty.city_id
7	INNER JOIN country AS ctr ON cty.country_id = ctr.country_id
8	LEFT JOIN
9	(SELECT A.customer_id,
10	A.first_name,
11	A.last_name,
12	C.city,
13	D.country,
14	SUM(amount) AS total_amount_paid
15	FROM payment E
16	INNER JOIN customer A ON E.customer_id = A.customer_id
17	INNER JOIN address B ON A.address_id = B.address_id
18	INNER JOIN city C ON B.city_id = C.city_id
19	INNER JOIN country D ON C.country_id = D.country_id
20	WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule (Dhulia)',
21	'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
22	GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country
23	ORDER BY total_amount_paid DESC
24	LIMIT 5)
25	AS top_5_customers

Data Output	Messages	Notifications
country character varying (50)	all_customer_count bigint	top_customer_count bigint
1 Mexico	30	2
2 India	60	1
3 United States	36	1
4 Turkey	15	1

Top 5 cst's and country base:

"country"	"all_customer_count"	"top_customer_count"
"Mexico"	30	2
"India"	60	1
"United States"	36	1
"Turkey"	15	1

Step 3:

- 1- I believe step 1 and 2 could be done without using subqueries, however this would be more time consuming and costly as well, which ends up being inefficient. Since we have two different queries, we could run them separately, and then connect results, however depending on the size or complexity of your database, this task could still be nearly impossible, or result in a really demanding work. Also, this data becomes more rigid as you would either need to run both queries again every time certain data changes, or you would have an outdated outcome based on initial results.
- 2- Subqueries are useful when there's a need to combine two different queries, where one result depends on the other outcomes. This means that we do not necessarily have both data on our original dataset, therefore both queries need to be connected and would run together, guaranteeing a accurate and up-to-date result considering any changes or any relevant data.